COSC204 Computer Systems

Lecture 13: Introduction and history of operating systems

Zhiyi Huang

School of Computing, University of Otago

General info

- Lecturer:
  Zhiyi Huang
  Office: 126 Owheo Building
  Email: `zhiyi.huang@otago.ac.nz`
- Recommended Textbooks:
  - For lectures: *Operating System Concepts with Java* (10th Edition), by A. Silberschatz, J. Galvin and G. Gagne, published by John Wiley
  - For labs: *Advanced Programming in the UNIX Environment* (3rd Edition), by W. Richard Stevens, published by Addison Wesley
  - For extended reading: *Computer Systems: A Programmer's Perspective* (Global Edition or 3rd Edition), by R. Bryant and D. O'Hallaron, published by Pearson

Lectures

- Questions are greatly encouraged!
- Hands-on experience with real OSs and hardware is helpful.
- For those who are interested in a real OS, you may study Zhiyi's teaching OS `xv6`: https://github.com/zhiyihuang/xv6_rpi2_port. It will used to illustrate some concepts in lectures.
- Run OSs for free using virtual machines on an existing computer.
- Buy a Raspberry Pi computer to experience with Linux OS on real hardware if you don't want to install it on your own computer.

Labs and internal assessment

Labs are important part of the paper. They can help you understand the concepts, principles, and algorithms of OS better.

Summary of the internal assessment for OS part of COSC204:

- C programming test: 10%.
- Programming assignment: 15%.
- Total: 25% plus assessed labs.

COSC204 Course overview (for part 2)

- Introduction and history of OS: **Ch1 (1.1–1.8)**
- Processes and system calls: **Ch2 (2.1–2.7)**
- File systems **Ch10 (10.1–10.6), Ch11 (11.1–11.4)**
- Device drivers **Ch13 (13.1–13.5)**
- Processes communication **Ch3 (3.1–3.3)**
- Processes sharing and threads **Ch3 (3.4–3.5), Ch4 (4.1–4.4)**
- CPU scheduling **Ch5 (5.1–5.3, 5.7)**
- Synchronisation and deadlock **Ch6 (6.1–6.7), Ch7**
- Memory management **Ch8 (8.1–8.6)**
- Virtual memory **Ch9 (9.1–9.7)**
- I/O systems **Ch13 (13.1–13.5)**
- Protection of file systems **Ch10 (10.6)**
- Advanced topics in computer systems
- Wrap up

Why care about Operating Systems?

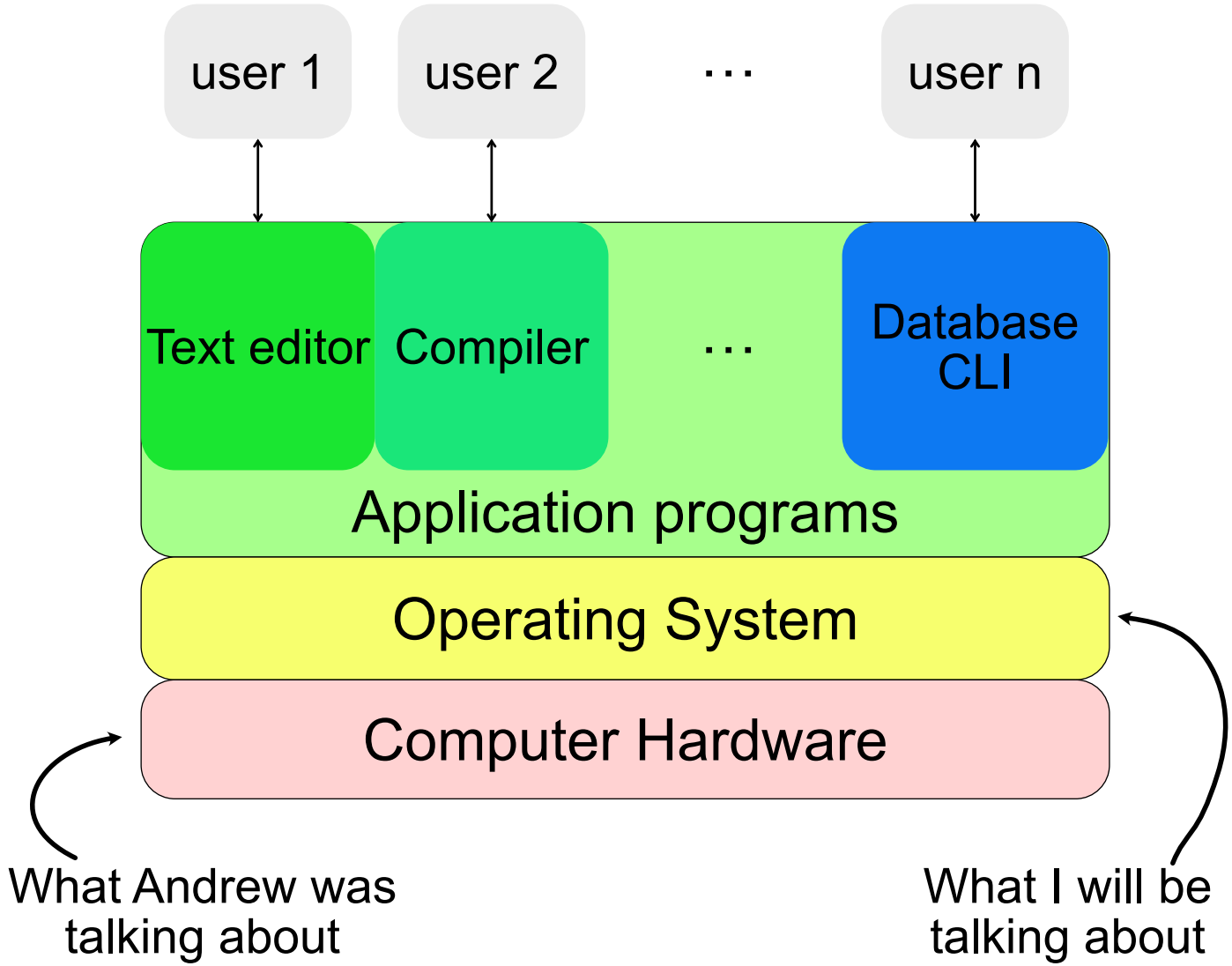
All your applications, games, etc., depend on the OS.

- Applications may crash if the developers do not understand the limitations of OS.
- Your programs may run suboptimally if you don't understand how OS works.
- Your software can have security flaws if you don't know how OS handles resources.

You have much more complete knowledge with OS experience.

- OS development involves leading developers.
- You will understand computer hardware and security better.
- Working with embedded systems and Internet of Things (IoT) demands skills in OS design and implementation.
- Co-design of hardware and software.

What is an Operating System?



What is an Operating System?

- OS is a program (also called the 'OS kernel') that is loaded into memory (RAM) very soon after the computer is powered on.
- OS kernel has highest privilege of all software and can access any hardware directly.
- User programs cannot access hardware directly, instead they make service requests of the OS kernel (system calls).
- User programs are called 'processes' when they are executed. A process is a program in execution. It is the basic execution entity (also called 'task' or 'job') managed by OS.

Kernel mode and user mode

CPU has two running modes: kernel and user.

- Software running in kernel mode has the privilege to access any hardware.
- Software running in user mode cannot access hardware directly (unless permitted otherwise by the OS kernel).
- The CPU has a status bit that tracks whether it is running software in kernel mode or user mode.
- OS kernel changes the bit when it switches control to non-privileged software.
- When an interrupt or system call occurs, the bit is set to kernel mode, and control passes to OS software.

Kernel mode and user mode

To be safe, keep ‘the kids’ out of ‘the kitchen’!





What is an Operating System?

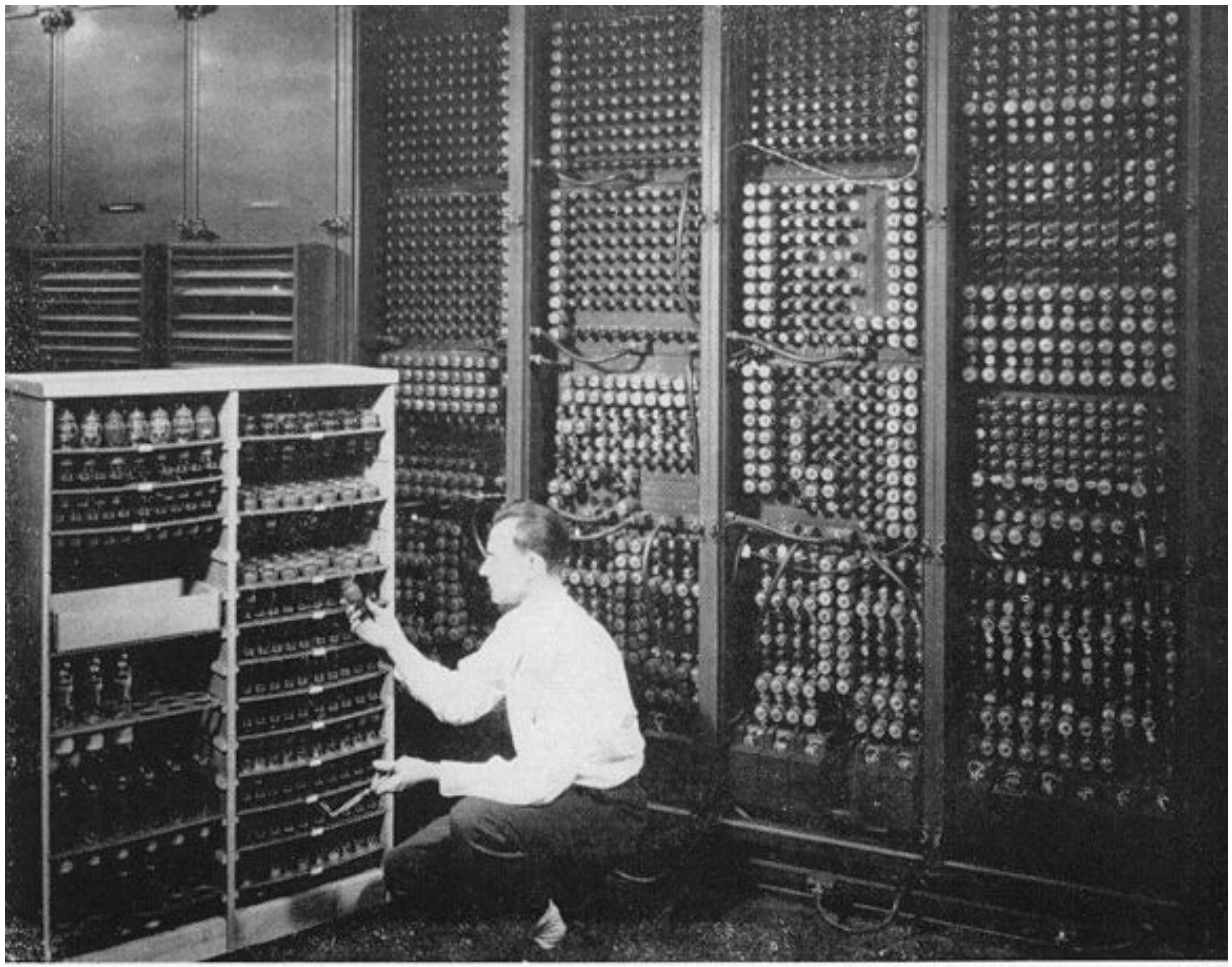
Let's define the functions of an OS:

An operating system (kernel) is a program that *manages* the different aspects of the operation of the machine and runs with the highest privilege in a CPU-protected domain.

- Manages processes running on the machine.
- Manages data storage like RAM and file systems on the machine.
- Manages input/output devices associated with the machine.
  - Manages networks and communications with other machines.
  - Manages the user interface.
- Manages security and protection enforcement and violations.

ENIAC was completed in 1945

First programmable, electronic, general-purpose, digital computer.

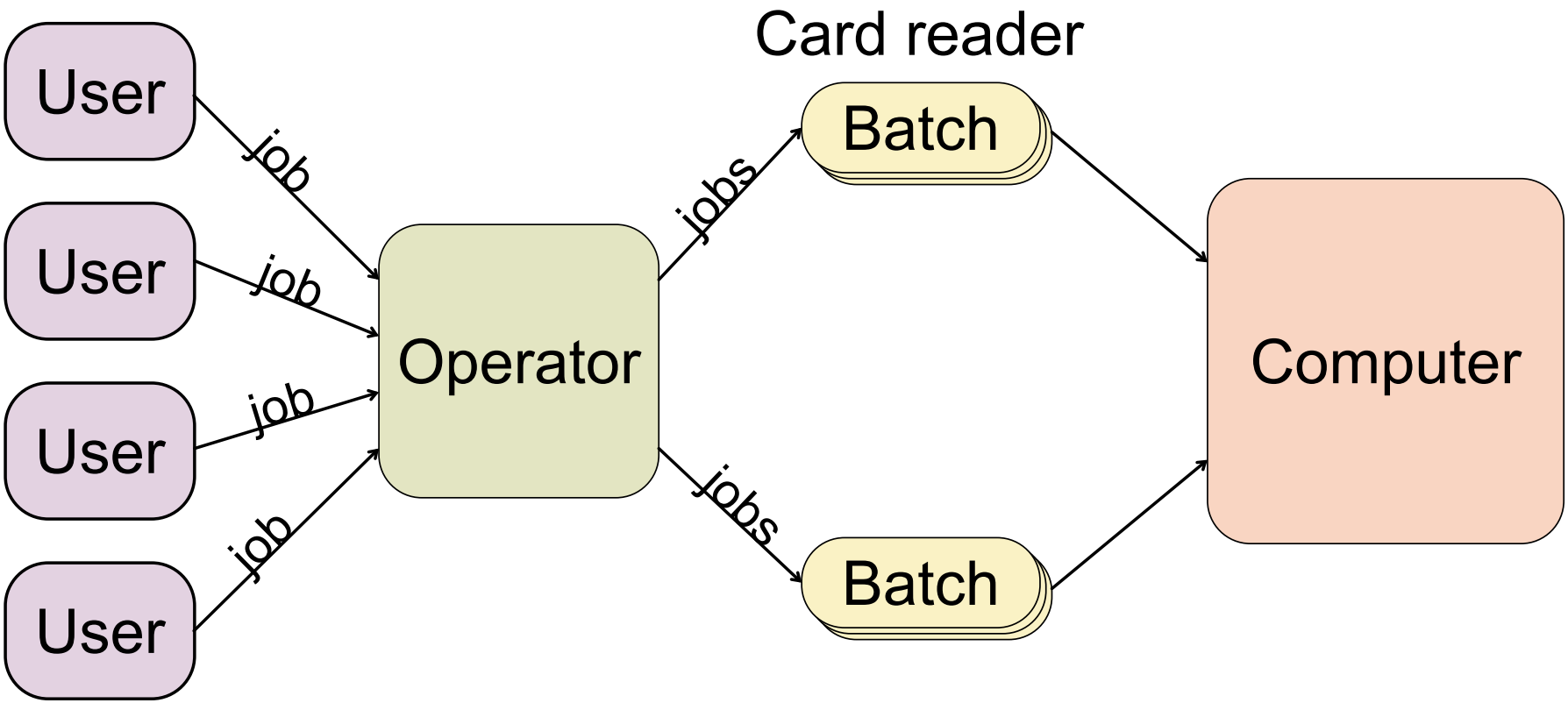


Replacing a bad tube meant checking among ENIAC's 19,000 possibilities.

OS evolution through history

- No OS.
- Batch systems.
- Spooling systems.
- Time sharing systems.
- Real-time systems

Batch operating systems



Batch operating systems

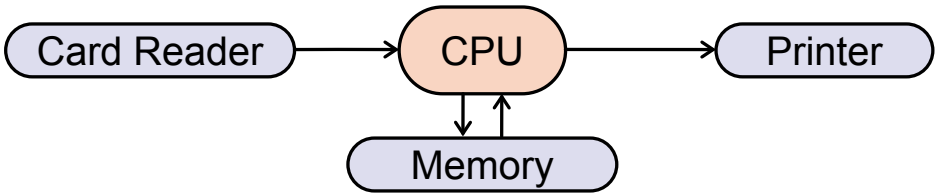
The first operating systems were introduced to increase the efficiency of (I/O) processes.

- Programmers didn't deal directly with these machines: instead, they prepared their programs as 'jobs', and submitted their jobs to *operators*. It wasn't possible to interact with a program while it was executing.
- Operators grouped jobs together and submitted batches of jobs to the machine. The computer ran operating system software to transfer control automatically from one job to the next.

These early batch operating systems were still inefficient:

- I/O processing is still very slow in comparison to CPU processing;
- While I/O was occurring, the CPU was often idle.

Spooling (Simultaneous Peripheral Operations On Line)



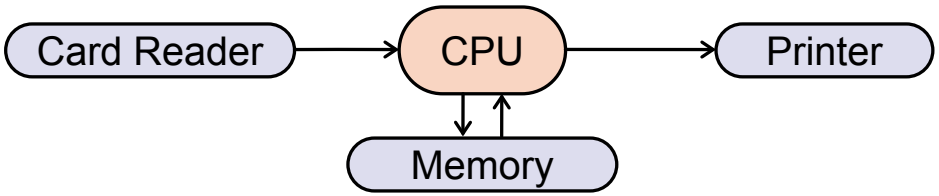
Spooling (Simultaneous Peripheral Operations On Line)

A solution to the I/O bottleneck is to perform I/O for one job J_1 *at the same time as* the CPU is executing another job J_2. This is known as spooling.

In this scenario, we need a secondary storage medium:

- somewhere to store the jobs being input to the system;
- somewhere to store the output from jobs in the system.

The secondary storage medium is a disk.



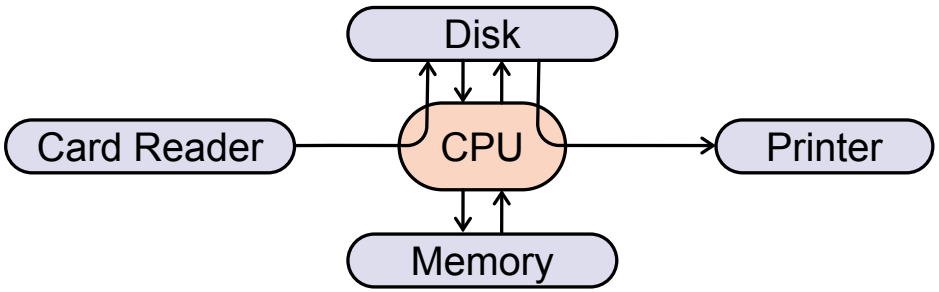
Spooling (Simultaneous Peripheral Operations On Line)

A solution to the I/O bottleneck is to perform I/O for one job J_1 *at the same time as* the CPU is executing another job J_2. This is known as spooling.

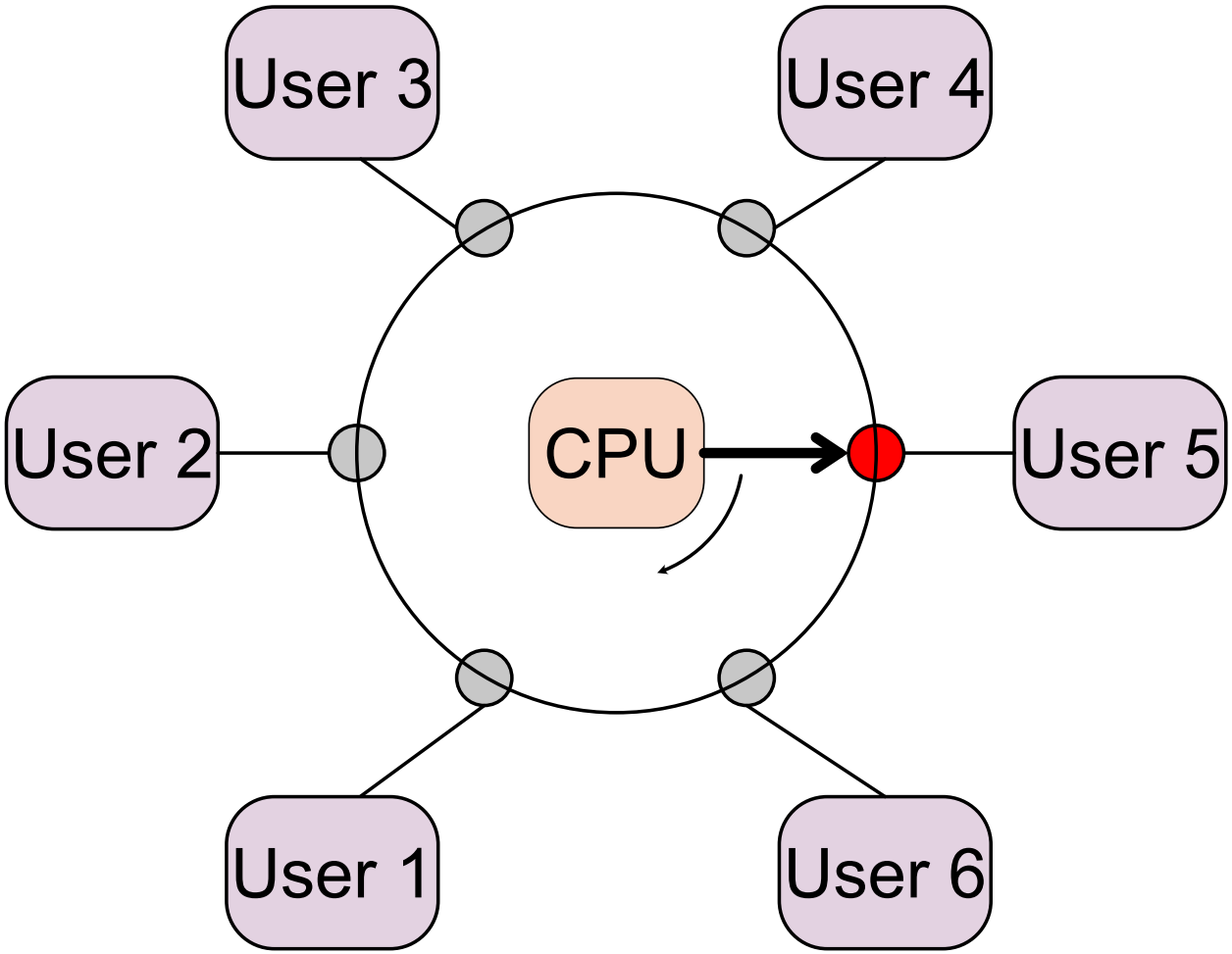
In this scenario, we need a secondary storage medium:

- somewhere to store the jobs being input to the system;
- somewhere to store the output from jobs in the system.

The secondary storage medium is a disk.



Time-sharing operating systems



Time Sharing OS

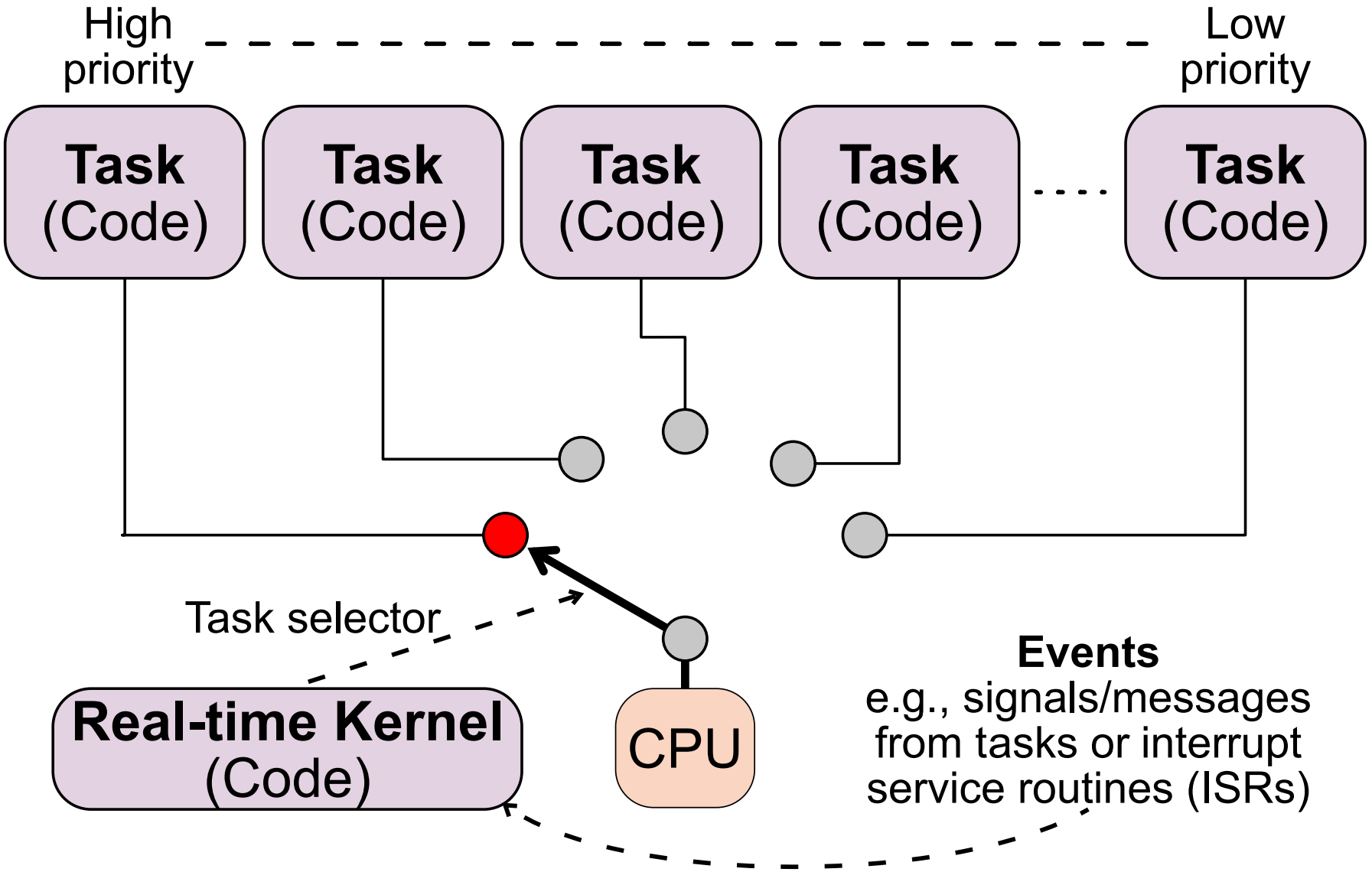
So far we have just been considering batch (i.e. non-interactive) programs. However, the concept of multiprogramming can be applied to interactive programs too.

- An interactive program is constantly checking for user input while it is running.
- If *several* interactive programs are multiprogrammed, and the CPU switches *quickly* between them, they behave as though they are being run independently, on several (slower) CPUs.

This type of system is called a time-sharing, or multitasking system.

Another reason for the big mainframes using time sharing is that CPUs were very expensive, and it paid to be able to share a single CPU among several users.

Real-time operating systems



Real-time systems

In some domains, there are rigid time requirements for processes to complete. For these domains, a real-time operating system is required.

- Hard real-time systems: guarantee that jobs be completed within specified times. (This typically means that advanced operating-system features are not provided: too risky.)
- Soft real-time systems: some jobs have higher priorities than others, and high priority jobs are processed preferentially.

Summary and readings

- Concepts: definition of OS, kernel and user modes
- Evolution of operating systems: batch systems, spooling systems, time sharing systems, real time systems, ...

- For this lecture, read Chapter 1 (Sections 1–8).